

PocketSmart AI

AI-Powered Smart Budget and Recommendation Assistant

Project Documentation: Structure, Functionality and Modules

1. Project Overview

PocketSmart AI is a complete FastAPI + Jinja2 + SQLite application that helps users plan spending and receive personalized recommendations for home interiors, parties and jewellery. It combines guided budget-planning forms, JWT authentication, recommendation history, optional Gemini-powered recommendation generation and a deterministic fallback catalog so the application can still work without an AI key.

Component	Technology
Backend	Python 3.11+, FastAPI, Uvicorn
Frontend	HTML, CSS, JavaScript, Jinja2 templates
Database	SQLite with SQLAlchemy
Authentication	JWT-based authentication
AI / Recommendation	Google Gemini via google-genai; deterministic catalog fallback

2. Problem Statement

Planning a purchase, home setup or event within a fixed budget can require comparing many products, services, platforms and price ranges. Users may spend significant time deciding how much to allocate to each category and which options fit their preferences. PocketSmart AI addresses this workflow by combining budget interpretation, domain-specific planning forms and AI-assisted recommendations in one application. The system is designed to reduce manual comparison while keeping the final choice with the user.

3. Project Objectives

- Provide budget-aware planning for Home Interior, Party and Jewelry use cases.
- Generate structured recommendations from user budgets, preferences and contextual inputs.
- Support optional outfit-image input for the Jewelry Planner.
- Provide user registration, login, JWT authentication and personalized recommendation history.
- Offer deterministic fallback recommendations so the application works without an AI key.

- Integrate Google Gemini through the official google-genai Python SDK for optional structured AI generation.
- Provide responsive Jinja2-based pages with HTML, CSS and JavaScript interactions.
- Provide automated API tests covering application health, authentication and planner workflows.
- Use destination search URLs and estimated prices instead of claiming unsupported live marketplace inventory, ratings or availability.

4. Key Use Cases

Use Case	User Input	Generated Output
Home Interior Planner	Budget, room type, item quantities, style/preferences	Budget-aware home decor, furniture and accessory recommendations
Party Budget Planner	Budget, guest count, party type, venue type and party needs	Structured event plan with food, venue, decoration and related suggestions
Jewelry Planner	Budget, occasion, style preferences and optional outfit image	Style-aware jewelry recommendations matched to the provided context
Recommendation History	Authenticated user and previous planner requests	Saved recommendation history for later review and reuse

5. Technology Stack

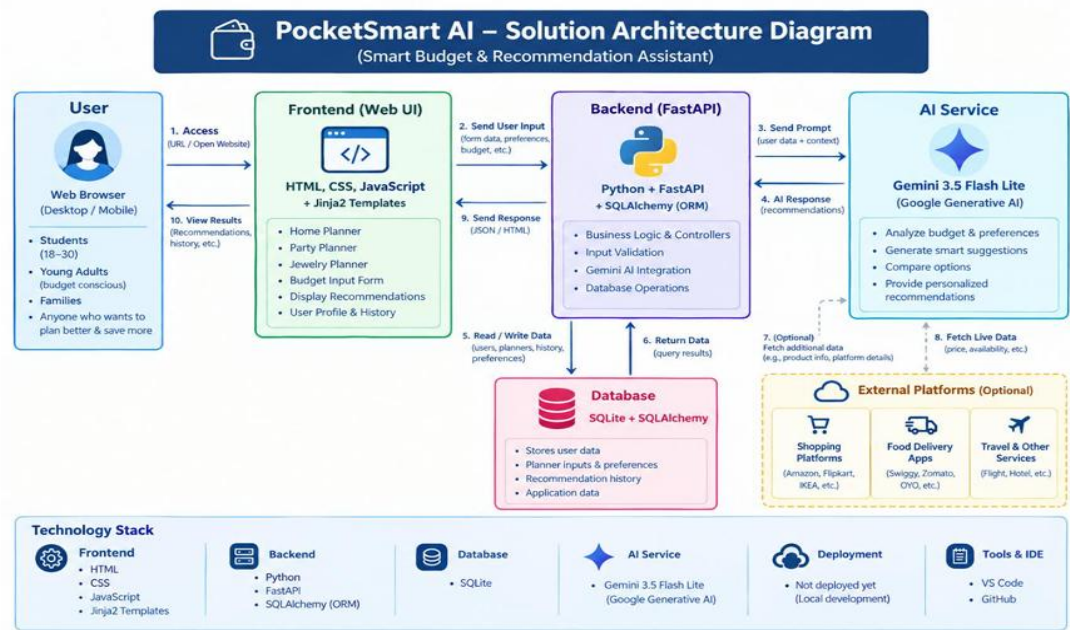
Layer	Technology	Purpose
Frontend	Jinja2 + HTML/CSS/JavaScript	Responsive planner forms, navigation and recommendation display
Backend	FastAPI	REST API, routing, request handling and application services
AI Layer	Google Gemini + google-genai	Optional structured recommendation generation from user context
Database	SQLite + SQLAlchemy	User accounts, recommendation records and application persistence
Authentication	JWT	Secure login and protected planner/history access
Validation	Pydantic	Validation and structured handling of API inputs/outputs

Fallback Engine	Deterministic catalog	Reliable recommendations when AI is disabled or unavailable
Testing	Pytest	Automated API and workflow verification
Runtime	Python 3.11+ recommended	Application development and execution

6. System Architecture

- The system follows a modular web-application architecture. The browser presents Jinja2-rendered pages and JavaScript interactions. FastAPI receives planner and authentication requests, validates the input and coordinates the recommendation service. The recommendation layer can use Gemini for structured generation or fall back to the deterministic catalog. SQLite stores application data such as users and recommendation history.
- Frontend Layer — Jinja2 templates with responsive HTML/CSS/JavaScript for forms, navigation and recommendation cards.
- API Layer — FastAPI routes for authentication, planner generation, user information and history.
- Authentication Layer — JWT-based registration, login and protected API access.
- Recommendation Layer — Domain-specific recommendation logic for Home, Party and Jewelry planners.
- AI Core — Google Gemini integration using structured JSON output for reliable recommendation rendering.
- Fallback Layer — Deterministic catalog used when AI is disabled, unavailable or unsuccessful.
- Data Layer — SQLite with SQLAlchemy for persistent users and recommendation history.

6.1 Solution Architecture



7. Project Structure

The project is organized into separate modules so authentication, routing, recommendation logic, templates, static assets and persistence can be maintained independently.

PocketSmart-AI/

```
|
| ├── app/
| | ├── main.py
| | ├── config.py
| | ├── database.py
| | ├── models.py
| | ├── schemas.py
| | ├── security.py
| | ├── routes/
| | | ├── auth.py
| | | ├── pages.py
| | | └── planners.py
| | └── services/
| |   ├── catalog.py
| |   ├── gemini.py
| |   └── recommendations.py
| ├── static/
| | ├── css/app.css
| | └── js/
| └── templates/
├── tests/
├── .env.example
├── .gitignore
├── pytest.ini
├── requirements.txt
└── README.md
```

8. Core Functionalities

- User registration and login with JWT authentication.
- Home Interior Planner for budget-aware furniture, decor and related recommendations.
- Party Budget Planner for event planning based on budget, guest count, party type, venue and needs.
- Jewelry Planner with optional outfit-image upload for style-aware recommendations.
- Recommendation history for authenticated users.
- Structured Gemini output for consistent recommendation cards when AI integration is enabled.
- Deterministic fallback catalog so the application continues to work without an AI key.
- Destination search URLs for supported marketplaces and services without claiming live inventory.
- Estimated-price labeling so users are not misled about live pricing or availability.

- Responsive frontend built with Jinja2, HTML, CSS and JavaScript.

Home Budget Planner

Attribute	Details
File	app/routes/planners.py
AI model	Gemini or deterministic catalog
Endpoint	POST /api/generate-home

Generates budget-aware home interior recommendations based on the user's budget, room requirements and preferences.

- Collects home and room planning inputs through the Jinja2 interface.
- Sends validated planner data to the recommendation service.
- Returns structured recommendations with estimated prices and destination search links.

Party Budget Planner

Attribute	Details
File	app/routes/planners.py
AI model	Gemini or deterministic catalog
Endpoint	POST /api/generate-party

Creates an event plan around the user's budget, guest count, party type, venue type and party needs.

- Accepts total budget and number of guests.
- Supports event details such as party type and venue type.
- Produces organized recommendations and estimated spending allocations for the event.

Jewelry Budget Planner

Attribute	Details
File	app/routes/planners.py
AI model	Gemini or deterministic catalog
Endpoint	POST /api/generate-jewelry

Provides jewelry recommendations using budget, occasion and style preferences, with optional outfit-image input.

- Accepts jewelry planning preferences and optional image upload.
- Uses contextual recommendation generation when Gemini is enabled.

Returns recommendation cards and destination search links while treating displayed prices as estimates.

Recommendation Service

Attribute	Details
File	app/services/recommendations.py and app/services/gemini.py
AI model	Gemini + deterministic catalog fallback
Endpoint	Used by planner endpoints

Central service that produces structured recommendations for the three planner domains.

- Uses Google Gemini through the google-genai SDK when AI mode is enabled.
- Requests structured JSON output so recommendation cards can be rendered consistently.
- Uses the deterministic catalog when AI generation fails or AI mode is disabled.
- Does not claim live inventory, live prices, ratings or availability for providers that are not connected through authenticated APIs.

Backend API

Attribute	Details
File	app/main.py and app/routes/
AI model	None (routing layer)
Endpoint	All API endpoints

Exposes RESTful FastAPI endpoints and connects the planner, authentication, database and recommendation layers.

- Provides registration, login and current-user endpoints.
- Provides Home, Party and Jewelry recommendation-generation endpoints.
- Provides recommendation-history access for authenticated users.

Serves the Jinja2 frontend and static files.

Frontend

Attribute	Details
File	app/templates/ and app/static/
AI model	None
Endpoint	Calls all planner and authentication endpoints

Provides the responsive web interface through which users interact with PocketSmart AI.

- Jinja2 templates provide the landing page, dashboard and planner pages.
- Separate planner forms collect budget and contextual information for Home, Party and Jewelry use cases.
- JavaScript submits forms to the backend and displays recommendation results.
- CSS provides responsive layouts, navigation, buttons, cards and result sections.

9. Running the Project

- Install Python 3.11+ and the dependencies listed in requirements.txt.
- Create and activate a virtual environment, then install the project dependencies.
- Copy .env.example to .env and configure the application settings.
- For the easiest first test, set USE_AI=false so the deterministic fallback catalog is used.

To enable AI recommendations, set USE_AI=true and provide GEMINI_API_KEY and the configured Gemini model in .env.

Start the server with: `uvicorn app.main:app --reload`

- Open **`http://127.0.0.1:8000`** in a browser.
- Run automated tests with: **`pytest -q`**
- Use `http://127.0.0.1:8000/docs` to test the FastAPI endpoints interactively.

10. Project Screenshots

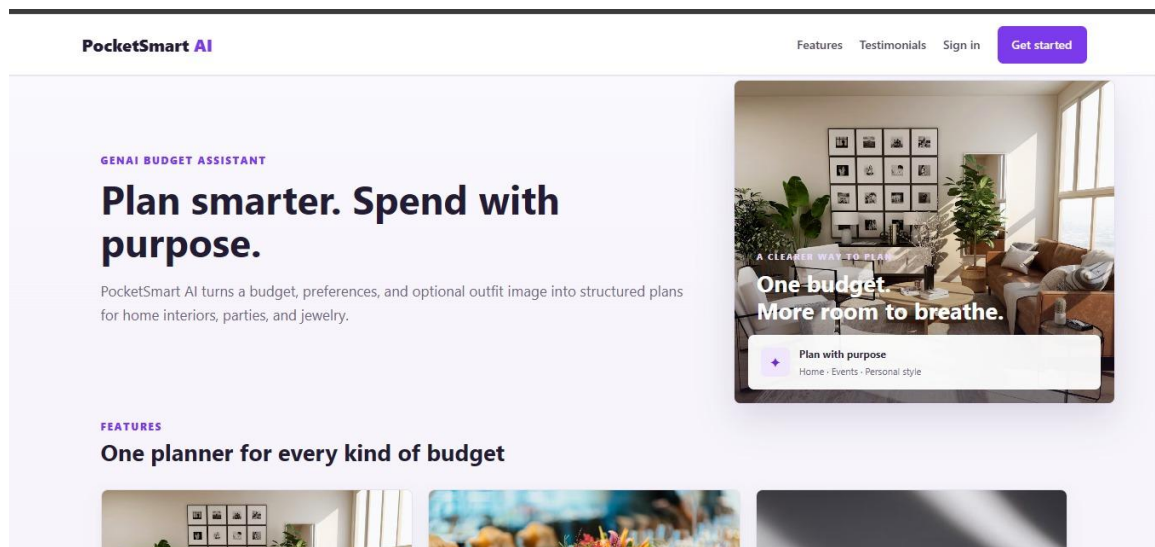


Figure 1. PocketSmart AI Home Page

The screenshot shows the PocketSmart AI Home Budget Planner interface. The header includes the logo "PocketSmart AI" and navigation links for "Dashboard", "Home Budget Planner", "Party Budget Planner", "Jewelry Budget Planner", "History", and "Logout". The main content area features the heading "HOME BUDGET PLANNER" and the subheading "Make your home budget go further". Below this, a paragraph states: "Set your budget, choose the rooms, and tell us what you need." The form is divided into two main sections: "Budget Details" and "Fixtures & Furniture". The "Budget Details" section has a "Total Budget (₹)" input field with a placeholder "e.g. 50,000". The "Fixtures & Furniture" section has four input fields: "Number of Lights / Fixtures", "Number of Ceiling Fans", "Number of Furniture Pieces", and "Number of Dining Tables", each with a placeholder "0".

Figure 2. Home Budget Planner

PocketSmart AI

Dashboard

Home Budget Planner

Party Budget Planner

Jewelry Budget Planner

History

Logout

PARTY BUDGET PLANNER

Plan an event that fits your budget

Shape an event plan around your guests, venue, and priorities.

Basic Information

Total Budget (₹)

e.g. 50,000

Number of Guests

e.g. 50

Event Details

Party Type

Birthday

Venue Type

Home

Party Needs

Figure 3. Party Budget Planner

PocketSmart AI

Dashboard

Home Budget Planner

Party Budget Planner

Jewelry Budget Planner

History

Logout

JEWELRY BUDGET PLANNER

Find a look for your occasion

Get jewelry recommendations shaped by your budget and personal style.

Budget Details

Total Budget (₹)

e.g. 5,000

Occasion & Preferences

Occasion

Birthday

Metal Preference

Any

Style Preferences

Describe your style preferences, materials, colors, etc.

Figure 4. Jewelry Budget Planner

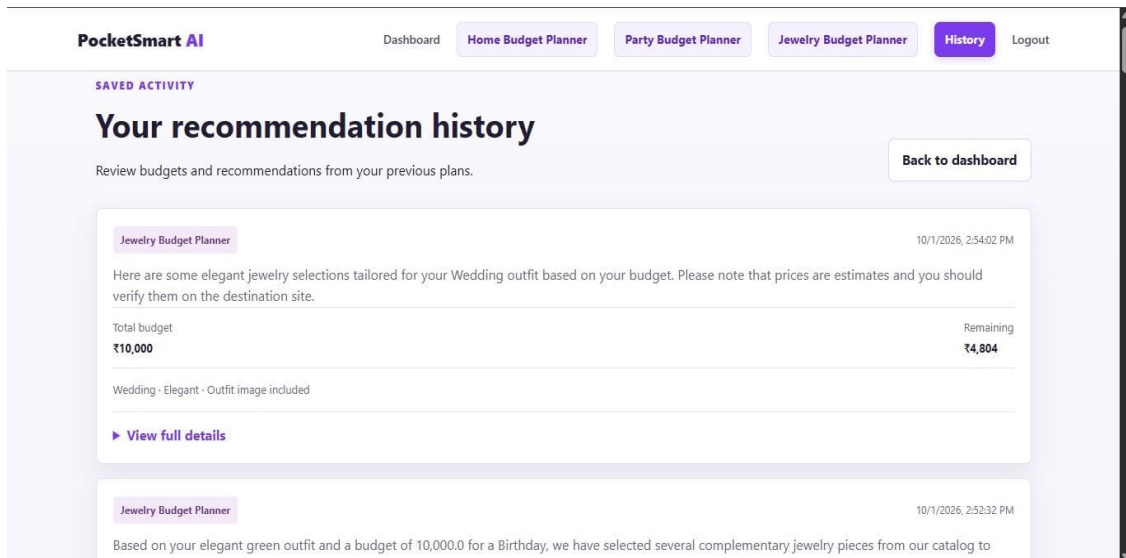


Figure 5. Recommendation History

11. ADVANTAGES & DISADVANTAGES

Advantages:

- One application supports multiple budget-planning domains.
- Personalized recommendations based on user budget and preferences.
- Optional multimodal input for jewellery planning.
- FastAPI provides a modular backend architecture.
- JWT authentication and recommendation history support personalized use.
- Deterministic fallback recommendations allow operation without an AI key.
- Responsive Jinja2 frontend is suitable for browser-based use.

Disadvantages:

- Marketplace results are destination search links rather than verified live inventory.
- Prices are estimates and may differ from actual provider prices.
- AI quality depends on the configured Gemini model and available API quota.
- SQLite is mainly suitable for development and smaller deployments; production systems may require a scalable database.

12. Future Enhancements

- Integration with official marketplace and service APIs when authenticated API access is available.
- Real-time price and availability synchronization where provider contracts permit it.
- More planner categories such as travel, groceries, education and personal finance.
- Improved multimodal recommendations for additional product categories.
- Enhanced multilingual interaction and recommendation presentation.
- Advanced budget optimization across multiple categories and constraints.
- Cloud deployment with scalable database and backend infrastructure.
- Improved recommendation analytics, saved plans and versioned planning history.

13. Conclusion

PocketSmart AI demonstrates how Generative AI can be combined with a modern Python web stack to simplify budget planning and personalized recommendations. The project brings together FastAPI for backend processing, Jinja2 for the user interface, SQLite for persistence, JWT for authentication and Google Gemini for optional structured AI generation.

The system supports Home Interior, Party and Jewelry planning while maintaining a deterministic fallback path so the application remains functional without an AI key. Its use of estimated prices and destination search URLs also avoids presenting unsupported live marketplace information as factual. With official API integrations, richer planning domains, multilingual capabilities and production deployment, PocketSmart AI can be extended into a broader budget-aware personal planning assistant.

14. Project Deliverables

- FastAPI-based PocketSmart AI application.
- Jinja2, HTML/CSS and JavaScript frontend.
- SQLite database with SQLAlchemy persistence.
- JWT-based user registration and authentication.
- Home Interior Budget Planner.
- Party Budget Planner.
- Jewelry Planner with optional image upload.
- Gemini-based structured recommendation generation.
- Deterministic fallback catalog for AI-independent operation.
- Recommendation history for authenticated users.
- Automated API tests using pytest.
- Local setup and execution workflow with Uvicorn.
- Source code organized into modular routes, services, models, templates and static assets.

15. Project Links

GitHub Repository: https://github.com/Rohitha-19/PocketSmart-AI_NM-Project

Project Demo Video: https://drive.google.com/drive/folders/15py1MV06ZY8_cG_r9_0pC2kU9ps8Dub